CS 449 Sprint 1 Report
Peg Solitaire BrainVita

Dhulipalla Devan sai Chowdary| 1640909

GitHub repository: https://github.com/devansai10/cs449-peg-solitaire

For Sprint 2, I completed the playable version of Peg Solitaire BrainVita in Java. I used Swing for the interface and JUnit 5 for testing. A player can choose the board type and size, start a new game, make legal moves, and see when the game is over. I kept the game rules separate from the interface so I could test the logic without opening the GUI.

Demonstration video: https://youtu.be/dz_-Ajgh2Rs?is=JQyxv_6x2HA_IAPg

1 Demonstration:

I will use the following order in my video so I can show the main features and the test results within the three-minute limit.

Time	Feature	What to demonstrate
0:00-0:30	Choose settings	Select English, Hexagon, and Diamond; select sizes 7 and 9.
0:30-0:55	Start New Game	Start a chosen layout and show the empty center and zero moves.
0:55-1:40	Make moves	Complete a legal move and attempt an invalid move that changes nothing.
1:40-2:15	Game over	Show a terminal game and explain the remaining-peg rating.
2:15-3:00	Automated tests	Run mvn clean test and show 16 passing tests and BUILD SUCCESS.

2 Summary of Source Code

My project has four production classes and three test classes. The line counts below include code, comments, and blank lines.

Source code file	Classification	Lines
BoardShape.java	Production	45
PlayerRating.java	Production	17
SolitaireGame.java	Production	123
SolitaireApp.java	Production	190
PlayerRatingTest.java	Test	40
SolitaireGameTest.java	Test	113
Sprint2AcceptanceTest.java	Test	108
TOTAL	375 production + 261 test	636

Design Responsibilities

Class	Responsibility
BoardShape	Defines playable positions for English, Hexagon, and Diamond layouts.
SolitaireGame	Owns board state, move validation, move execution, legal-move search, and game-over detection.
PlayerRating	Converts a positive remaining-peg count to Outstanding, Very Good, Good, or Average.
SolitaireApp	Builds the Swing interface, reads player selections, starts games, and renders model state.

3 Production Code vs User Stories and Acceptance Criteria

I organized the Sprint 2 requirements into four user stories. For each acceptance criterion, I listed the classes and methods that implement it.

- **User Story 1 Choose the Board Size and Type**

As a player, I want to choose the board size and type so that I can play on my preferred layout.

AC	Given When Then	Implementation	Status
1.1	Given the settings are displayed, When I select English, Hexagon, or Diamond and size 7 or 9, Then my choices appear and are used when I click New Game.	SolitaireApp.show(); BoardShape subclasses	Complete
1.2	Given a game is in progress, When I change the selected size or type without clicking New Game, Then the active board, peg positions, peg count, and move count stay unchanged.	SolitaireApp.show() New Game listener	Complete

- **User Story 2 Start a New Game**

As a player, I want to start a new game with my chosen size and type so that I can play with those settings.

AC	Given When Then	Implementation	Status
2.1	Given I selected a supported board type and size, When I click New Game, Then a board with those settings appears with a peg in every playable hole except the center, no pegs outside the layout, and zero moves.	SolitaireApp.show(); SolitaireGame constructor; BoardShape.contains()	Complete

AC	Given When Then	Implementation	Status
2.2	Given the current game has moves, a selected peg, or a final result, When I click New Game, Then a fresh board uses the current settings, the move count resets to zero, and the selection and old result are cleared.	SolitaireApp.show(); SolitaireApp.update(); SolitaireGame constructor	Complete

- **User Story 3 Make a Move in a Game**

As a player, I want to move a peg according to the game rules so that I can remove pegs and progress through the game.

AC	Given When Then	Implementation	Status
3.1	Given the game is in progress and, at any point during play, a peg has one adjacent peg and an empty playable hole exactly two positions away in the same horizontal, vertical, or diagonal direction, When I select the starting peg and destination, Then the move is accepted, the source and jumped holes become empty, the destination contains the moving peg, the peg count decreases by one, and every other position stays unchanged.	SolitaireGame.isLegal(); SolitaireGame.move(); SolitaireApp.select()	Complete
3.2	Given the selected move does not jump over exactly one adjacent peg into an empty playable hole two positions away in an allowed direction, When I attempt the move, Then the move is rejected and the board, peg count, and move count remain unchanged.	SolitaireGame.isLegal(); SolitaireGame.move(); SolitaireApp.select()	Complete

- **User Story 4 Determine When the Game Is Over**

As a player, I want the game to detect when play is over so that I can see my final rating.

AC	Given When Then	Implementation	Status
4.1	Given a legal move will leave one peg, When I complete that move, Then the game ends and displays one peg remaining with the rating Outstanding.	SolitaireGame.isOver() SolitaireGame.result() PlayerRating.rate() SolitaireApp.update()	Complete
4.2	Given more than one peg remains, When no legal horizontal, vertical, or diagonal move is available anywhere, Then the game ends and displays Very Good for 2 pegs, Good for 3 pegs, or Average for 4 or more pegs.	SolitaireGame.legalMoves() SolitaireGame.isOver() SolitaireGame.result() PlayerRating.rate()	Complete
4.3	Given more than one peg remains and at least one legal move is available, When the game checks for another move, Then the game continues and no final rating is displayed.	SolitaireGame.legalMoves() SolitaireGame.isOver() SolitaireGame.result()	Complete

4 Tests vs User Stories and Acceptance Criteria

I ran the full test suite with Maven and JUnit 5 on October 27, 2026. All 16 tests passed. There were no failures, errors, or skipped tests.

4.1 Automated Tests Directly Corresponding to Acceptance Criteria

AC ID	Test class	Test method	Status
1.1, 2.1	SolitaireGameTest	everyLayoutStartsWithOnlyItsCenterEmpty	Pass
1.2, 2.1, 2.2	Sprint2AcceptanceTest	restartingWithSelectedBoardResetsMovesAndCenter	Pass
3.1	SolitaireGameTest	orthogonalMoveRemovesExactlyTheJumpedPeg	Pass
3.1	SolitaireGameTest	diagonalMoveIsAllowedByTheCourseRules	Pass

AC ID	Test class	Test method	Status
3.2	SolitaireGameTest	invalidMovesDoNotChangeBoardOrMoveCount	Pass
3.2	Sprint2AcceptanceTest	invalidJumpLeavesEveryCellUnchangedAndAllowsRecovery	Pass
4.1	Sprint2AcceptanceTest	onePegFinishIsOutstanding	Pass
4.1	PlayerRatingTest	onePegReceivesOutstanding	Pass
4.2, 4.3	Sprint2AcceptanceTest	deterministicGameEndsAndRejectsFurtherMoves	Pass
4.2, 4.3	SolitaireGameTest	completeGamesTerminateAndKeepValidCountsAcrossAllLayouts	Pass

4.2 Manual Tests Directly Corresponding to Acceptance Criteria

ID	AC	Input	Expected result	Result
MT-01	1.1, 1.2	Change type and size during play; click New Game only after checking active state.	Selection changes first; active board changes only after New Game.	Pass
MT-02	2.1, 2.2	Start a chosen game, make a move, then click New Game.	Chosen layout has empty center and zero moves; restart clears prior state.	Pass
MT-03	3.1, 3.2	Make one legal jump; then attempt an illegal jump.	Legal move removes one peg; illegal move leaves every value unchanged.	Pass
MT-04	4.1-4.3	Play to a terminal board and compare the displayed rating with peg count.	Game stops at one peg or no moves; rating follows the required table.	Pass

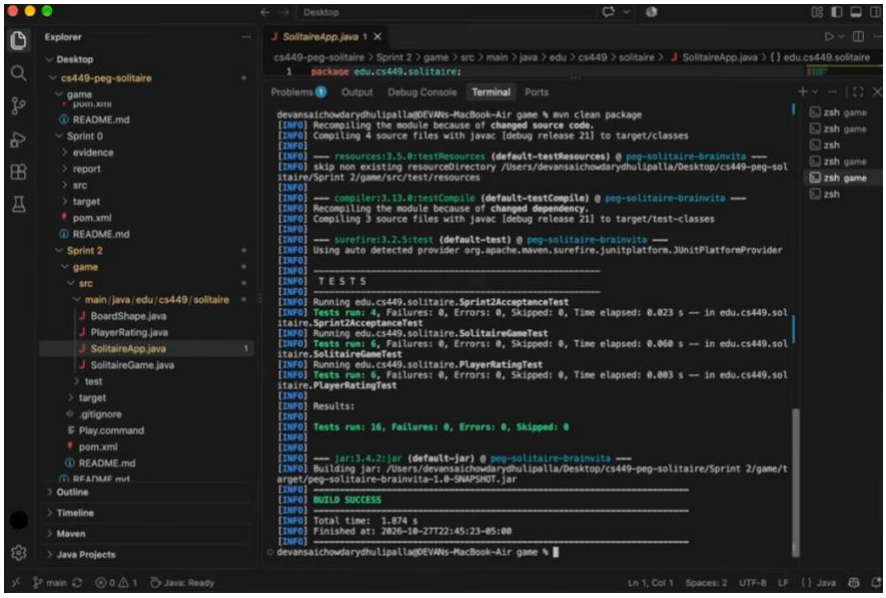
4.3 Other Automated Tests

Test	Input	Expected result	Class and method
Unsupported board sizes	-1, 0, 6, 8, 100	IllegalArgumentException	SolitaireGameTest.unsupportedSizesAreRejected
Rating boundaries	0 and -1 pegs	IllegalArgumentException	PlayerRatingTest.zeroPegsAreRejected; negativePegsAreRejected
Rating categories	1, 2, 3, 4, and 32 pegs	Correct required rating	PlayerRatingTest methods
Seeded complete games	60 games across all types and sizes	Termination and peg-count invariant	SolitaireGameTest.completeGamesTerminateAndKeepValidCountsAcrossAllLayouts

5 Verification Evidence

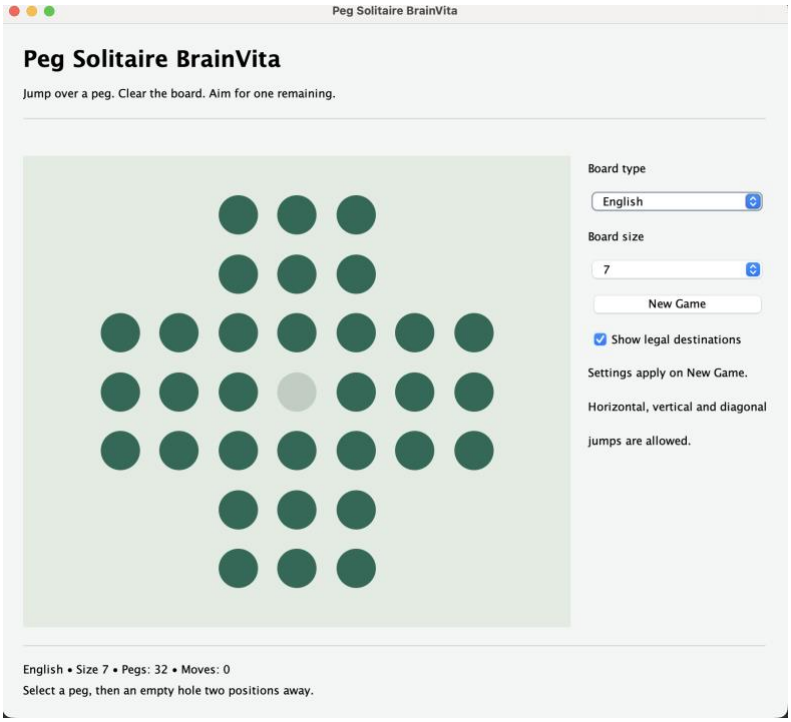
5.1 Maven and JUnit Results

I ran mvn clean test with Java 21 after finishing the Sprint 2 changes. Maven compiled all four production files and all three test files. The results are shown below.



5.2 Graphical Interface

The GUI lets the player choose a board type and size, start a new game, and turn move hints on or off. It also shows the peg count, move count, and current status. I kept the same board interaction from Sprint 1 and added the Sprint 2 tests and report mappings.



6 AI Assisted Unit Test Evidence

I used ChatGPT to draft two JUnit tests and then checked both answers against the actual public API. The first is a regression test for the existing New Game behavior and uses Diamond only as an already-supported board choice; it is not presented as new Sprint 2 functionality. The second directly supports acceptance criterion 1.2 by confirming that an invalid move leaves every board cell and both counters unchanged. The first response contained incorrect method names, so I supplied the exact API and refined the answer before running the complete test suite.

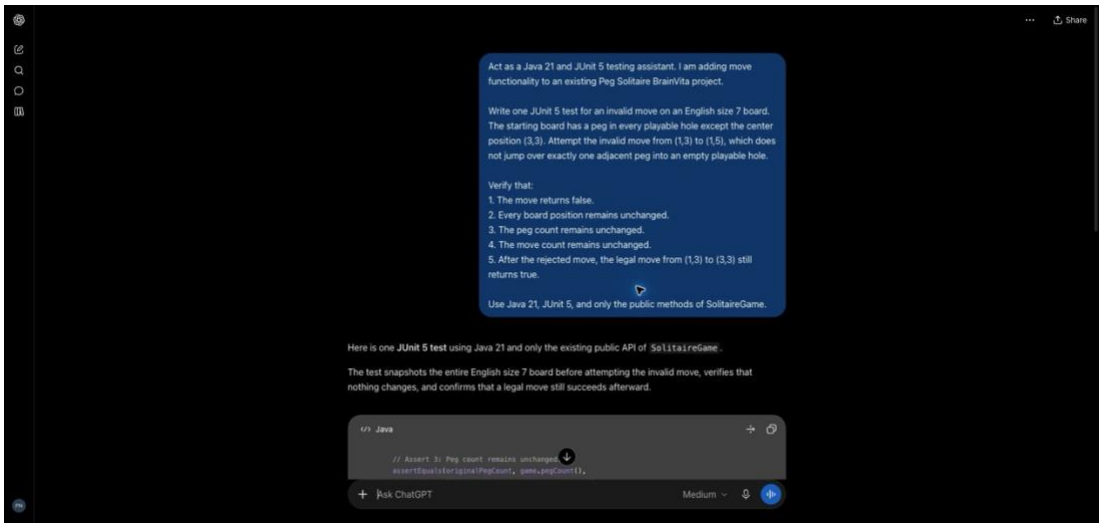


Figure 3. First ChatGPT prompt and the beginning of its generated New Game test.

6.1 Error Found and Corrected

ChatGPT's first answer used BoardShape.ENGLISH, BoardShape.DIAMOND, getPegCount(), getMoveCount(), getResult(), and move(int, int, int, int). Those names do not exist in my code. I corrected the answer by providing the actual constructors and methods: new BoardShape.English(), new BoardShape.Diamond(), pegCount(), moveCount(), result(), and move(Move). The revised test uses the real API.

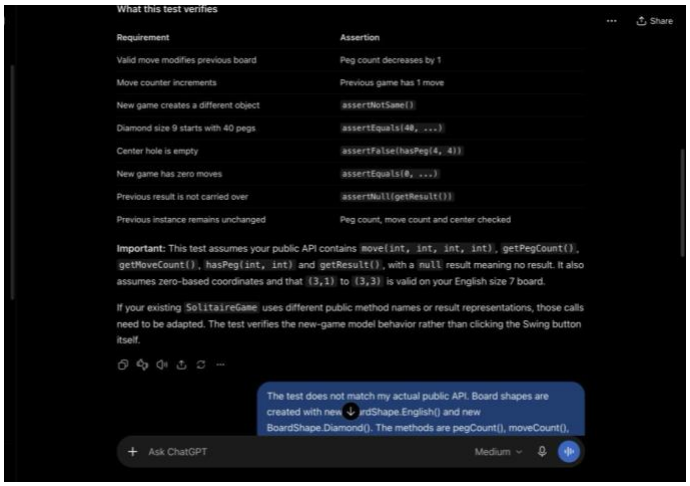


Figure 4. I identified the API errors and asked ChatGPT to rewrite the first test with the exact public methods.

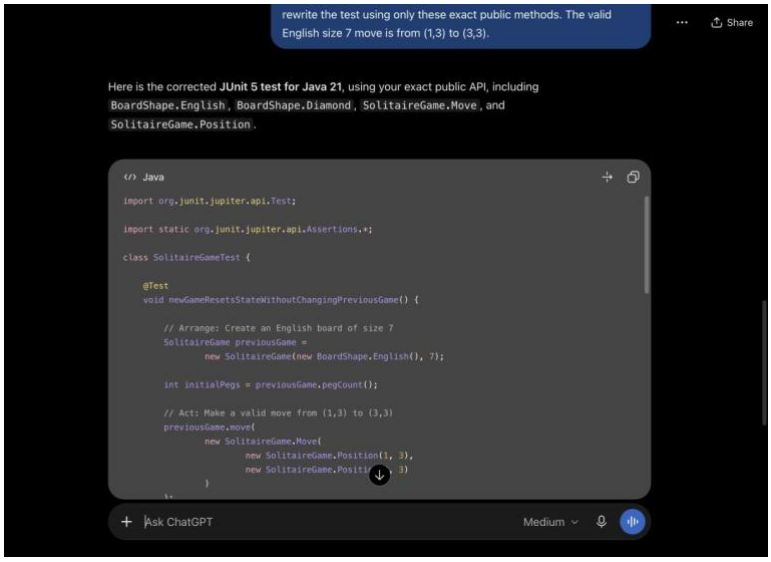


Figure 5. ChatGPT's revised answer uses the project's actual BoardShape, Move, Position, and counter APIs.

6.2 Second AI Generated Test

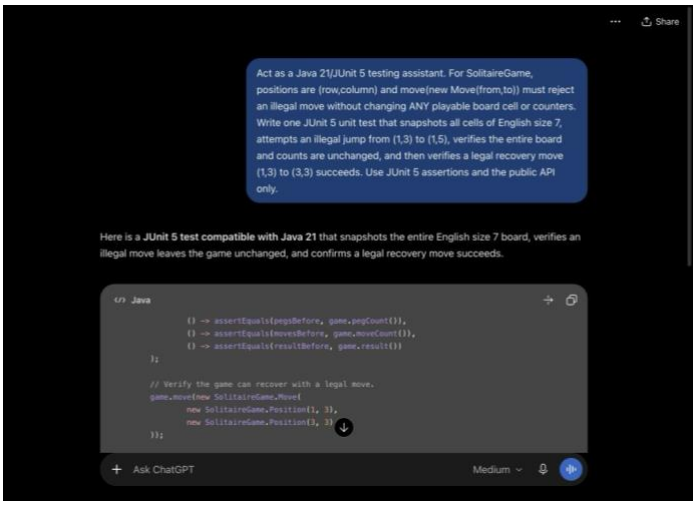


Figure 6. Second ChatGPT prompt for invalid-move state preservation and legal recovery.

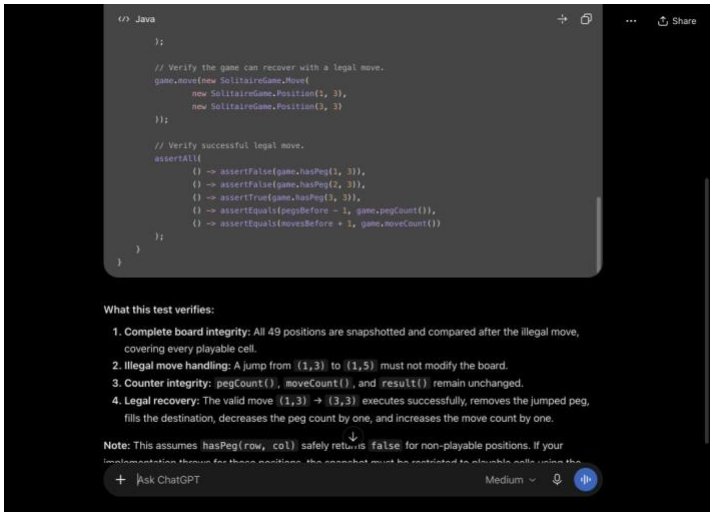


Figure 7. The second answer checks every board cell, the counters, and the legal recovery move.